

Multi-User Social VR for Parent Group Discussion

Media Upload Pipeline and Avatar/Facial System Design Document

Metadata	Value
To	Dr. Lei Zhang, Kennesaw State University
From	Mason Prather, Graduate Research Assistant
Last Updated	May 31, 2026
Revision Focus	Purpose/rationale, feature purpose, clearer diagram plan, and screenshots
Primary Demo Scene	Assets/_Scenes/PhonePhotoUpload.unity
Unity Version	6000.3.8f1

Purpose

This document describes the architecture and supervisor-review status of the Multi-User Social VR for Parent Group Discussion Unity project. The project contains two connected feature areas: a Quest-hosted phone media upload pipeline and an avatar/facial tracking system for embodied social presence. Both features are framed around the same research goal: supporting bereaved parents as they share memories, discuss their children, and participate in emotionally meaningful group discussion inside a shared immersive environment.

A. Quest user menu



B. Shared media display



Figure 1. Current visual evidence for the media upload workflow: the Quest user menu provides pairing instructions, and the uploaded image appears on the shared in-world media display.

Table of Contents

- 1. Executive Summary
- 2. Project Background and System Rationale
- 3. Project Areas
- 4. Document Scope
- 5. Current Project Status
- 6. Media Upload Pipeline Design Documentation
- 7. Media Upload Supervisor Review and Success Criteria
- 8. Media Upload Runtime Flow and Diagram Requirements
- 9. Media Upload Configuration, Storage, Logging, and Security
- 10. Avatar and Facial System
- 11. Dependencies and Repository Map
- 12. Known Limitations and Maintenance Guidance

1. Executive Summary

Multi-User Social VR for Parent Group Discussion is designed to support bereaved parents in a shared virtual reality space where they can remember their children, share personal stories, and participate in group discussion. The project therefore requires more than a basic VR scene. It needs systems that support emotional communication, embodied social presence, and personal memory sharing.

The current active supervisor-review deliverable is the Quest-based Media Upload Pipeline. In this workflow, the Quest hosts a local browser upload page, the headset displays a local upload URL and pairing code, a phone browser on the same local network submits an image, the upload is stored under `Application.persistentDataPath/Uploads`, and the newest image appears inside the Unity/Quest scene. This feature allows parents to upload meaningful photos from their own devices and use those images as part of storytelling and remembrance during group discussion.

The project also contains an earlier Avatar and Facial System. That subsystem used Ready Player Me avatars, FinalIK/VRIK binding to headset and controller anchors, Quest Pro eye and facial expression sources, optional lip-sync, networked avatar experiments, and Vivox/social presence support. Its purpose was to make participants feel represented as emotionally expressive people rather than static digital figures. For the current supervisor demo, this subsystem is documented but inactive because the project scope shifted to the media upload workflow, and the RPM/FinalIK/Quest Pro tracking stack added dependency, access/licensing, maintenance, and validation overhead that is not required for the active demo.

Together, these two project areas support the core design goal: enabling bereaved parents to communicate naturally, share memories visually, and feel socially present with one another in an immersive VR environment.

2. Project Background and System Rationale

The Wind Phone Therapy VR experience is intended for emotionally sensitive group discussion with bereaved parents. Participants may want to speak about their children, share memories, listen to others, and respond in a way that feels personal and socially present. This creates two major design needs: the system must let participants bring personal media into the space, and it must help participants appear as expressive social beings in the space.

2.1 Purpose of the Overall System

The overall system is meant to support parent group discussion in VR by combining a shared immersive environment, expressive avatars, and personal photo sharing. The VR environment gives participants a common place to meet. The avatar/facial system gives participants an embodied identity and supports nonverbal emotional cues. The media upload system lets participants bring personally meaningful images into the conversation from their phones.

2.2 Purpose of the Media Upload Feature

The media upload feature exists so parents can upload photos or memory-related images from their personal devices and display them inside the VR environment. For a bereavement-centered discussion, this matters because memories are often attached to specific images. A participant may be able to speak more clearly and personally when a photo is visible to the group. The system therefore supports storytelling, remembrance, and shared discussion rather than acting as a generic file-transfer tool.

2.3 Purpose of the Avatar and Facial Tracking Feature

The avatar and facial tracking feature exists to increase social presence and emotional communication. In a sensitive group discussion, participants may rely on facial expressions, gaze direction, head movement, hand gestures, and posture to communicate empathy, attention, and emotional response. The avatar system was designed to capture those signals from Quest Pro/OVR tracking sources and apply them to a full-body avatar representation.

2.4 Relationship Between Both Feature Areas

The media upload system and the avatar/facial system are separate technically, but they support the same experience goal. The media upload system provides personal memory artifacts for discussion. The avatar/facial system provides embodied and emotional presence during that discussion. The intended user experience is a parent entering VR, appearing as an expressive avatar, uploading a meaningful image from a phone, and using that image while speaking with the group.

3. Project Areas

Project Area	Status	Purpose	Main Output
Project Area 1: Media Upload Pipeline	Active / primary supervisor demo	Browser-based phone-to-Quest media transfer for personally meaningful images.	Uploaded image appears in the Unity/Quest scene.
Project Area 2: Avatar and Facial System	Documented / inactive for current demo	Full-body and expressive avatar presence using RPM, FinalIK, Quest Pro tracking, optional networking, and Vivox/social presence support.	Local/remote avatars with IK-driven body pose and facial expression mapping when restored and validated.

4. Document Scope

- In scope: browser-based phone-to-Quest media upload, pairing-code workflow, persistent upload storage, Unity display bridge, supervisor review path, and documentation of the Avatar and Facial System developed during earlier project iterations.
- Avatar documentation scope: Ready Player Me avatar loading, FinalIK/VRIK embodiment, Quest Pro eye/facial tracking, local and network avatar setup, face/lip-sync/network replication experiments, and Vivox/social presence support where present in the repository.
- Out of active demo scope but documented for handoff: RPM avatar loading/embodiment, RPM face tracking, Vivox/social presence, broad multiplayer/template systems, phone mirroring/WebRTC experiments, and old app-side companion upload clients.
- Repository basis: ProjectSettings, Packages/manifest.json, Assets/_Scenes, Assets/_Scripts, Assets/FromCompanion, Assets/VRMPAssets, Assets/_Prefabs, README.md, and docs/.

5. Current Project Status

The status summary is split by project area so the active media upload deliverable is not confused with the documented avatar/social-presence architecture.

5.1 Media Upload Pipeline Status

System	Status	Notes
Phone Web Upload Page	Active / primary demo	Quest hosts the local browser upload page through M_ServerBootstrap and M_SimpleHttpServer.
Pairing Code Pipeline	Active / primary demo	M_ServerBootstrap generates or loads the code; M_SimpleHttpServer validates browser form submissions.
Phone Media Upload	Active / primary demo	Phone uses a standard browser on the same local network. No phone Unity companion app is required.
Uploaded Image Display	Active / primary demo	M_PhoneUploadToDisplay loads the newest saved upload and applies it to M_QuestPhotoDisplay.
Quest User Menu	Active / primary demo	Launch-visible/summonable headset UI displays the URL, pairing code, passthrough toggle, recenter action, and close button.
Multiplayer Shared Media Sync	Optional / patched; requires two-Quest validation	M_NetworkedPhotoSync can broadcast media through XRINetworkPlayer when an owning local network player is spawned.
Companion App Upload Flow	Deprecated early experiment	App-side client scripts remain for traceability only.
Phone Mirroring / WebRTC	Deprecated early experiment	Phone mirroring scripts remain in Assets/_Scripts/PhoneMirror and are not part of the current media upload demo.



Figure 2. Quest user menu showing local upload URLs, pairing code, passthrough toggle, recenter control, and close button.

5.2 Avatar and Facial System Status

System	Status	Notes
Ready Player Me Avatar Loading	Documented / inactive for current demo	RPM package and avatar scripts remain in the repository; avatar loading is not part of the active media upload path.
FinalIK / VRIK Avatar Embodiment	Documented / inactive for current demo	RootMotion FinalIK is present and M_LocalAvatarManager/M_NetAvatar configure VRIK targets when avatar workflows are
Local Avatar Setup	Documented / inactive for current demo	M_LocalAvatarManager loads an RPM avatar, parents it under a local avatar root, hides first-person head meshes, and binds OVR/XR anchors.
Network Avatar Setup	Documented / inactive for current demo	M_NetPlayer, M_NetPoseDriver, and M_NetAvatar provide avatar URL and head/hand pose replication paths.
Quest Pro Eye/Face Tracking	Documented / inactive for current demo	M_LocalFaceDriver reads OVRFaceExpressions and maps expression values to RPM-compatible blendshapes.
Vivox Voice / Social Presence	Retained / requires separate validation	Vivox package and scripts exist for social/avatar/multiplayer presence, but Vivox is not required for the media upload supervisor demo.



Figure 3. Avatar/facial tracking visual evidence showing an expressive avatar in the shared VR environment. This figure should be placed in the Avatar and Facial System section to connect the technical subsystem to the social-presence purpose.

6. Media Upload Pipeline Design Documentation

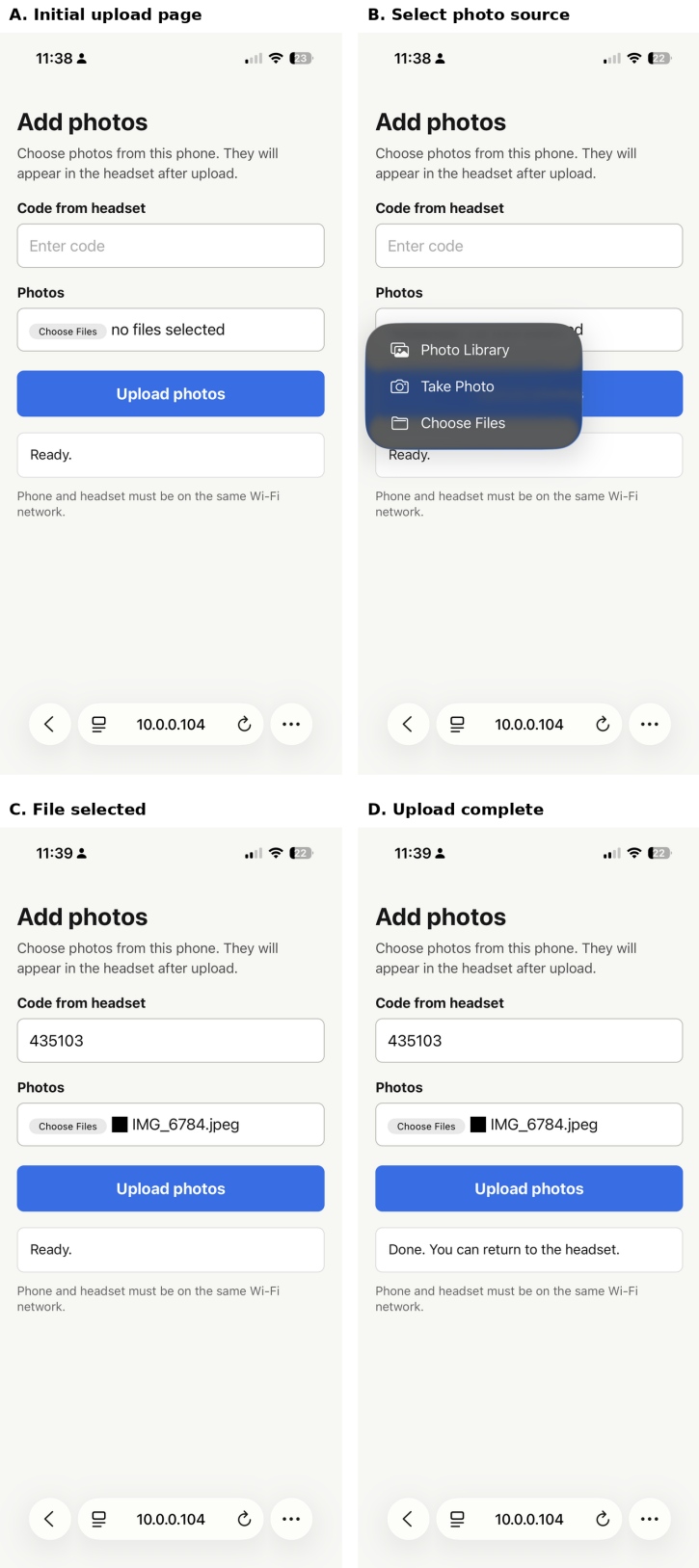
The Media Upload Pipeline is the current active deliverable. Its design keeps the phone side lightweight: the Quest hosts the page and endpoint, while the phone acts only as a local-network browser client.

6.1 System Purpose

The pipeline supports no-install phone-to-Quest image transfer for supervisor review and future parent group discussion. It lets a reviewer or participant open the active scene, use the launch-visible headset menu, read the displayed URL and pairing code, upload a phone image through a browser, and see that image appear in the Unity/Quest scene.

For the parent group discussion use case, this feature is specifically intended to let parents bring photos of their children or other meaningful memory images into the VR environment. This allows personal media to become part of the conversation rather than forcing participants to rely only on verbal description.

Figure 4. Phone browser upload workflow: initial page, photo selection, selected file state, and successful upload confirmation.



6.2 Active Components

Component	Active Role
M_ServerBootstrap	Starts the HTTP server, prepares persistent upload storage, resolves local network addresses, generates or loads the pairing code, publishes headset instructions, and starts UDP discovery beacons.
M_SimpleHttpServer	Serves the browser upload page and handles /upload-photo, /ping, and /status.
M_QuestUserMenu	Presents the launch-visible and summonable headset-side menu for phone pairing instructions, passthrough, recentering, and dismissal.
M_PhoneImportHeadsetMode	Supplies phone pairing text for the menu and retains optional prompt/pairing behavior for scenes that opt into it.
M_PhoneUploadToDisplay	Observes LastSavedPhotoPath, reads the saved image file, decodes it into a Texture2D, and passes it to M_QuestPhotoDisplay.
M_QuestPhotoDisplay	Applies the latest uploaded texture to a RawImage or renderer material and updates status/file labels.

6.3 User-Facing Media Upload Flow

1. The headset displays a Quest user menu with the local IP upload URL and a pairing code.
2. The participant opens the displayed URL on a phone connected to the same Wi-Fi/local network.
3. The participant enters the pairing code shown in the headset.
4. The participant chooses a photo from the phone photo library, camera, or file picker.
5. The phone submits the image through the Quest-hosted upload page.
6. The Quest saves the file and updates the in-world media display.
7. The participant can use the displayed image as part of storytelling and group discussion.



Figure 5. In-VR media uploader and shared media display. The left panel shows uploaded media thumbnails, and the right panel shows the currently shared image for discussion.

7. Media Upload Supervisor Review and Success Criteria

7.1 Recommended Supervisor Review Path

8. Open Assets/_Scenes/PhonePhotoUpload.unity.
9. Run the scene on the Quest/headset target or configured Quest runtime.
10. Confirm the Quest user menu opens automatically and displays a local upload URL and pairing code. If closed, reopen it with the left controller menu/start input.
11. Connect a phone to the same Wi-Fi/local network as the Quest.
12. Open the displayed local IP URL in the phone browser.
13. Enter the pairing code shown in the headset.
14. Select and upload a non-sensitive test image.
15. Confirm Unity saves the uploaded image under Application.persistentDataPath/Uploads.
16. Confirm the newest image appears in the VR scene display.
17. Check Unity console logs from M_ServerBootstrap, M_SimpleHttpServer, M_QuestUserMenu, M_PassthroughModeController, and M_PhoneUploadToDisplay.
18. Do not use RPM avatars, phone mirroring/WebRTC, Vivox, or old companion app flows as the main media upload demo path.

7.2 Demo Success Criteria

Criterion	Expected Evidence
Headset displays upload URL	Quest user menu shows at least one local IP upload URL.
Headset displays pairing code	Quest user menu shows the active code for the current scene run.
Phone browser loads Quest-hosted page	Phone opens the displayed URL on the same local network.
Phone image upload succeeds	Browser upload returns success after the code is entered.
Upload is saved to persistent storage	Server logs or file inspection show storage under Application.persistentDataPath/Uploads.
Image appears in Unity/Quest scene	Newest uploaded image is visible on the scene display target.
Unity logs show pipeline events	Logs show server startup, upload receipt, saved path, and display update.

8. Media Upload Runtime Flow and Diagram Requirements

All diagrams should be replaced with simple PowerPoint shape diagrams rather than raw text diagrams. Each diagram should use readable labels, one clear arrow direction, and a caption that explains the purpose of the diagram for the parent group discussion system.

8.1 Media Upload System Design Diagram - Required Content

Diagram Shape	Label / Explanation
Phone Browser	Opens Quest local URL and submits pairing code plus selected image.
Quest User Menu	Displays URL, pairing code, passthrough toggle, recenter, and close controls.
Quest-Hosted Web Upload Server	M_ServerBootstrap and M_SimpleHttpServer host the page and validate uploads.
Persistent Upload Storage	Application.persistentDataPath/Uploads stores submitted images.
Unity Upload Display Bridge	M_PhoneUploadToDisplay reads the newest saved image and decodes it.
VR Scene Display	M_QuestPhotoDisplay applies the texture to the shared in-world display.

Recommended caption: The phone acts as a no-install browser client while the Quest hosts the upload page, validates the pairing code, stores the image, and displays the newest uploaded media inside the VR scene.

8.2 Runtime Data Flow

19. Phone browser opens local Quest URL.
20. Phone submits pairing code and image POST.
21. M_SimpleHttpServer receives /upload-photo.
22. Server validates pairing code.
23. Server saves image under Application.persistentDataPath/Uploads.
24. M_SimpleHttpServer.LastSavedPhotoPath updates.

25. M_PhoneUploadToDisplay detects the saved path.
26. Unity reads bytes, decodes Texture2D, and applies it through M_QuestPhotoDisplay.
27. The uploaded image appears on the in-world media display for group discussion.

8.3 Optional Multiplayer Shared Media Flow

Shared media sync is separate from the core phone-browser upload demo, but it is intended to let multiple connected Quest Pro users see the same selected image. The diagram should show encoded image bytes, not Unity texture references or local file paths, because each client reconstructs and applies its own runtime texture.

Diagram Shape	Label / Explanation
Local User	Selects or imports an image and updates the local display.
M_NetworkedPhotoSync	Encodes image bytes and initiates broadcast through the owning local player.
XRINetworkPlayer	Chunks payload, reconstructs image, caches latest shared image, and relays to clients/host.
Remote Clients	Receive onSharedMediaReceived, decode bytes, and update remote M_QuestPhotoDisplay.
Late Joiners	Clear stale display state, then receive cached latest shared image if one exists.

9. Media Upload Configuration, Storage, Logging, and Security

9.1 Configuration

Item	Value	Notes
Unity version	6000.3.8f1	Verified from ProjectSettings/ProjectVersion.txt.
Primary scene	Assets/_Scenes/PhonePhotoUpload.unity	Primary active scene for supervisor review.
HTTP upload port	8080	M_ServerBootstrap.httpPort default.
UDP discovery port	7777	M_ServerBootstrap.discoveryPort default; discovery broadcasts PHOTO_SERVER beacons.
Upload root	Application.persistentDataPath/Uploads	Created by M_ServerBootstrap and used by the HTTP server/display bridge.
Maximum upload size	64 MB default	M_ServerBootstrap.maxUploadMegabytes default.
Pairing code	Required by default	M_ServerBootstrap.requirePairingCode default is enabled.
Legacy raw upload compatibility	Disable for demos unless intentionally testing compatibility	Keeps supervisor review focused on pairing-code enforcement.

9.2 Storage and Logging

Uploaded media is stored in Application.persistentDataPath/Uploads. The main diagnostic logs use [M_ServerBootstrap], [M_SimpleHttpServer], [M_QuestUserMenu], [M_PassthroughModeController], [M_PhoneUploadToDisplay], and [M_QuestPhotoDisplay] prefixes. Optional shared-media logs can also appear under [M_NetworkedPhotoSync] and XRINetworkPlayer when multiplayer sync is active.

9.3 Security, Privacy, and Network Assumptions

- The phone upload server is intended for trusted local lab networks only.
- Phone and Quest must be on the same local network, and that network must permit peer-to-peer traffic.
- The pairing code is lightweight access control, not full authentication.
- The system should not be used on public/open Wi-Fi without additional safeguards.
- Uploaded media should be cleared or controlled according to project review needs.
- Demo media should avoid sensitive participant images unless approved by the supervisor and any applicable study protocol.
- Deprecated phone mirroring/WebRTC scripts should not be treated as secure production infrastructure.

10. Avatar and Facial System

10.1 System Purpose

The Avatar and Facial System was designed to support embodied multi-user VR presence. It used Ready Player Me avatars as user avatar assets, FinalIK/VRIK to bind avatars to XR headset and controller targets, Quest Pro/OVR expression sources for face and eye-related expression mapping, and Netcode/Vivox components for networked avatar presentation and social presence.

For the parent group discussion use case, the purpose of this subsystem is emotional communication. Facial expressions, gaze direction, head motion, hand movement, posture, and voice presence help participants feel that they are speaking with emotionally present people rather than static representations.

10.2 Current Status

This subsystem is documented but inactive for the current media upload deliverable. It is retained for technical handoff, future reference, and repository traceability. It should not be presented as part of the current supervisor demo unless RPM loading, FinalIK/VRIK binding, Quest Pro tracking, Netcode state, and Vivox behavior are separately restored and validated.

10.3 Avatar System Components

Script / Asset	Former Role
M_LocalAvatarManager	Loads local RPM avatar, parents it under local avatar root, binds VRIK to OVR/XR anchors, hides first-person head meshes, and initializes face diagnostics.
M_NetPlayer	Stores owner-written network avatar URL in a Netcode NetworkVariable.
M_NetPoseDriver	Publishes owner head/hand pose NetworkVariables and applies smoothed pose values to remote IK targets.
M_NetAvatar	Loads networked RPM avatar, wires VRIK to replicated IK targets, binds face mesh to M_NetFaceMirror, and suppresses owner-local renderers.
M_LocalFaceDriver	Maps OVRFaceExpressions values to RPM/ARKit-style blendshape weights with gain, smoothing, and neutral relaxation.
M_NetFaceMirror	Samples owner local face weights, sends compact snapshots through RPCs, and applies smoothed remote weights.
M_OVRLipSyncAutoBinder	Maps OVR lip-sync visemes and laughter to RPM blendshape indices.
M_FaceDebugProbe	Collects neutral-expression diagnostics and logs blendshape summaries for calibration.
M_VivoxManager / VoiceChatManager	Manages Unity Services/Vivox channel membership, microphone permission, participant voice state, and positional voice settings.

10.4 Avatar Loading and FinalIK Design

M_LocalAvatarManager uses ReadyPlayerMe.Core AvatarObjectLoader with ARKit and Oculus Visemes morph targets. It reads PlayerPrefs RPM_URL or a serialized fallback URL, loads the avatar, parents the resulting GameObject under the local avatar root, sets animator behavior, optionally hides first-person head/eye/teeth/hair meshes, and configures VRIK targets from OVRCameraRig anchors or XR Origin-style fallbacks.

10.5 Eye and Facial Tracking Design

M_LocalFaceDriver reads OVRFaceExpressions values and maps them to RPM/ARKit-style blendshape names for jaw, mouth, cheeks, eyelids, eye-look directions, brows, and related expression channels. The driver applies configurable global gain, per-channel gain, smoothing, maximum weight clamps, per-channel cutoffs, and neutral-pose relaxation.

M_FaceDebugProbe collects neutral samples and logs min/max/average blendshape behavior for calibration.

M_OVRLipSyncAutoBinder can map OVR lip-sync visemes and laughter to RPM blendshape indices when a lip-sync workflow is restored.

10.6 Networked Avatar and Face Sync Design

M_NetPlayer stores the replicated RPM avatar URL, M_NetPoseDriver publishes owner head/hand poses through Netcode NetworkVariables, and M_NetAvatar loads each avatar and binds VRIK to replicated IK targets. M_NetFaceMirror samples owner-side local face blendshape weights, scales and thresholds them, sends snapshots through RPCs, and applies smoothed weights to network avatar face meshes.

10.7 Avatar and Facial System Deprecation Justification

The Avatar and Facial System is inactive for the current deliverable because the project scope shifted to the media upload workflow, and the RPM/FinalIK/Quest Pro face-tracking stack introduced dependency, access/licensing, maintenance, and validation overhead that was not required for the active supervisor demo. This deprecation is project-specific and should not be read as a universal claim that Ready Player Me, FinalIK, or Unity avatar workflows are unusable in all contexts.

11. Dependencies and Repository Map

11.1 Dependency Posture

Dependency Area	Current Posture	Notes
Meta XR SDK / OVR / XR Interaction Toolkit	Active Quest runtime support and historical Quest Pro tracking support	Supports current headset runtime/passthrough work and earlier OVRFaceExpressions avatar experiments.
UGUI and TextMesh Pro	Active UI support	Upload prompt, status labels, browser instructions, and display UI.
.NET sockets / Unity networking modules	Active support	Custom HTTP server uses sockets; Unity networking modules support runtime network behavior.
NativeGallery	Supporting / optional	Used by headset gallery import and old companion client image-picking paths.
Netcode and Unity Services Multiplayer	Optional for media upload; avatar/network replication support in earlier project area	Supports optional shared media and earlier network avatar pose/face replication.
Ready Player Me, glTFast, DracoUnity, RootMotion FinalIK	Avatar/facial system dependency set; inactive for current media upload demo	Required only if avatar workflows are reactivated.
Vivox	Avatar/social presence support; requires separate validation	Not required for the media upload demo.
Unity WebRTC	Early phone-interaction experiment dependency	Retained for phone mirroring/WebRTC prototype, not active media upload behavior.

11.2 Repository Source Map

Path	Role
Assets/_Scenes/PhonePhotoUpload.unity	Primary active scene for supervisor review and only enabled build scene.
Assets/_Scenes/EyeFaceTrackingScene.unity and variants	Earlier avatar/face tracking and mixed experiment scenes; disabled in build settings.
Assets/FromCompanion/Scripts/Server/	Active Quest-side upload server scripts and server-scene diagnostics.
Assets/_Scripts/MediaUpload/ and Assets/_Scripts/Menus/	Active display bridge, photo display, Quest user menu, passthrough support, optional gallery import, and optional network sync bridge.
Assets/_Prefabs/MediaUploadSetup.prefab	Scene-referenced media display and gallery prefab.
Assets/_Scripts/Player/	Avatar and facial system scripts for RPM loading, VRIK binding, pose sync, face mapping, and lip-sync helpers.
Assets/_Scripts/Network/	Project network helpers, including FaceSyncData and M_VivoxManager.
Assets/_Prefabs/NetworkedPlayer.prefab	Project network avatar/player prefab containing avatar and face mirror components.
Assets/VRMPAssets/	Retained multiplayer/template room assets, managers, voice chat, and network player support.
Packages/manifest.json	Unity package dependencies.
docs/	Repository documentation and this design document.

12. Known Limitations and Maintenance Guidance

12.1 Media Upload Pipeline Limitations

- The upload server is intended for trusted local lab networks only.
- Phone and Quest must be on the same local network, and that network must permit peer-to-peer traffic.
- The pairing code is lightweight access control, not full authentication.
- Uploaded files remain in Application.persistentDataPath/Uploads until manually cleared or handled by future cleanup tooling.
- No automated Unity test suite or repository-level validation runner was found for the active workflows.
- Optional shared-media sync and gallery import require separate validation before being presented as active deliverables.

12.2 Media Upload Maintenance Guidance

- Treat M_ServerBootstrap, M_SimpleHttpServer, M_QuestUserMenu, M_PassthroughModeController, M_PhoneImportHeadsetMode, M_PhoneUploadToDisplay, and M_QuestPhotoDisplay as the primary active maintenance surface.
- Preserve serialized Unity references, scene paths, prefab paths, and public script names unless performing a planned migration.
- Re-test upload URL display, code validation, file saving, texture decoding, and scene display after any upload pipeline change.
- Keep legacy raw upload compatibility disabled for code-enforced demos unless intentionally testing backward compatibility.
- Record Unity version, Quest device, phone OS/browser, Wi-Fi network, and Unity console logs when reporting issues.

12.3 Avatar and Facial System Limitations

- RPM/FinalIK/avatar tracking should not be presented as active without separate validation.
- Ready Player Me package/API availability, private avatar access, and licensing constraints must be checked before restoring avatar loading.
- FinalIK/VRIK, Quest Pro tracking, Vivox, and Netcode ownership behavior create a larger validation surface than the current media upload workflow requires.
- Face blendshape mapping depends on the selected avatar exposing compatible blendshape names and indices.
- Vivox/social presence requires Unity Services authentication, microphone permission, channel configuration, and device-level audio validation.